

PS 26052 — ANC Prototype Development

Software Implementation Task — 2-Module Structure

Build one continuous, modular software pipeline in which the output of each module becomes the input to the next.

Overview

This task consolidates the original five-stage software breakdown (S0–S4) into two implementation modules. The full technical pipeline — architecture, signal generation, discrete-time processing, statistical analysis, and Wiener/MMSE estimation — is preserved in its entirety; only the reporting structure changes. Each module now ends in one concrete, runnable deliverable, giving the team two clear checkpoints instead of five smaller hand-offs.

1. Pipeline

Module 1 — Architecture → Signal Generation →
Discrete-Time Processing
Module 2 — Correlation & Statistics → Wiener/MMSE
Estimation

The framework must remain extensible for later phases of the ANC roadmap:

LMS → NLMS → Secondary Path → FxLMS/FxNLMS → AI/Hybrid → Real-Time

Mapping from the original breakdown:

Original Stage	Consolidated Into
S0 – Project Architecture & Computational Framework	Module 1A
S1 – Discrete-Time Signal Generation & Visualization	Module 1B
S2 – Discrete-Time Systems & FIR/LTI Processing	Module 1C
S3 – Correlation & Statistical Signal Analysis	Module 2A
S4 – Wiener/MMSE Estimation	Module 2B

2. Notation

Symbol	Meaning
$x[n]$	Reference / input signal
$d[n]$	Desired signal

Symbol	Meaning
$h[n]$	FIR filter impulse response
$y[n]$	Filtered output signal
$R_{xx}[k]$	Autocorrelation of $x[n]$ at lag k
$R_{xd}[k]$	Cross-correlation of $x[n]$ and $d[n]$ at lag k
R	Correlation matrix (built from R_{xx})
p	Cross-correlation vector (built from R_{xd})
w_{opt}	Optimal Wiener filter coefficients
$\hat{d}[n]$	Wiener estimate of $d[n]$
$e[n]$	Error signal, $d[n] - \hat{d}[n]$
MSE	Mean-squared error

3. MODULE 1 – Signal Foundation: Architecture, Generation & Discrete-Time Processing

Objective: Stand up the reproducible software framework, then use it to generate and process discrete-time signals through an FIR/LTI system.

1A. Project Architecture & Computational Framework

Establish the software architecture and GitHub repository for the ANC project.

- Define the project directory and module structure.
- Define common signal, configuration, experiment, visualization, and result interfaces.
- Establish a reproducible software environment and dependency management.
- Create configuration-driven experiments — no hard-coded parameters.
- Create a common result-saving and visualization mechanism.
- Create the testing structure for future modules.

Output: Executable project framework, ready for the signal-processing modules.

1B. Discrete-Time Signal Generation & Visualization

Implement the computational representation of physical/time-varying signals as discrete-time sequences.

$$x[n] = x(nT_s), \quad T_s = 1/f_s$$

- Generate configurable sinusoidal, multi-tone, white-noise, colored-noise, impulse, and burst signals.
- Support configurable sampling frequency, duration, amplitude, frequency, and random seed.
- Use one common signal representation across every generated signal type.
- Visualize signals in the discrete-time domain, with basic frequency-domain visualization where applicable.

- Support arbitrary signal configurations without changing the implementation – this same generator produces both $x[n]$ and $d[n]$ for Module 2.

Output: Configurable discrete-time signal $x[n]$. **Feeds into:** 1C.

1C. Discrete-Time Systems & FIR/LTI Processing

Extend the signal framework to implement discrete-time linear systems and FIR filtering.

$$y[n] = \sum h[k]x[n-k]$$

- Implement configurable FIR filtering with arbitrary coefficients $h[k]$ and filter length.
- Represent and apply impulse responses.
- Demonstrate the relationship between input, system, and output.
- Visualize $x[n]$, $h[n]$, and $y[n]$ in the time domain, with frequency-domain visualization where applicable.

Output: Filtered signal $y[n]$ and system representation. **Feeds into:** Module 2.

Module 1 Deliverable: A configuration-driven framework that generates $x[n]$ and produces $y[n]$ through a defined FIR/LTI system, with full time/frequency-domain visualization – ready to supply Module 2's statistical analysis.

4. MODULE 2 – Statistical Estimation: Correlation Analysis & Wiener/MMSE Filtering

Objective: Characterize the statistical relationship between a reference and a desired signal, then solve for and validate the optimal (Wiener/MMSE) estimator.

2A. Correlation & Statistical Signal Analysis

Establish the statistical relationship between the reference signal $x[n]$ and a desired signal $d[n]$.

$$\begin{aligned} R_{xx}[k] &= E\{x[n]x[n-k]\} \\ R_{xd}[k] &= E\{x[n]d[n-k]\} \end{aligned}$$

- Calculate sample estimates of autocorrelation and cross-correlation.
- Construct the correlation matrix R and cross-correlation vector p required for finite-dimensional estimation.
- Support uncorrelated, correlated, and highly correlated signal cases.
- Visualize $x[n]$, $d[n]$, $R_{xx}[k]$, and $R_{xd}[k]$.
- Store statistical outputs in a reusable format for the estimation step.

Output: R_{xx} , R_{xd} , and p . **Feeds into:** 2B.

2B. Wiener/MMSE Estimation

Implement the finite-dimensional Wiener solution using the statistical quantities from 2A.

$$\begin{aligned} R w_{\text{opt}} &= p \\ w_{\text{opt}} &= R^{-1}p \end{aligned}$$

- Construct the Wiener correlation matrix R and cross-correlation vector p .

- Solve for the optimal coefficient vector w_{opt} .
- Generate the estimate and the error:

$$\begin{aligned}\hat{d}[n] &= w_{\text{opt}}^T x[n] \\ e[n] &= d[n] - \hat{d}[n]\end{aligned}$$

- Calculate empirical mean-squared error (MSE).
- Compare the Wiener result against a non-optimal initialization.
- Store coefficients, signals, and metrics as experiment results.

Required outputs: $x[n]$, $d[n]$, R , p , w_{opt} , $\hat{d}[n]$, $e[n]$, MSE

Required visualizations: reference/desired signals, estimated signal, error, correlations, Wiener coefficients, MSE

Module 2 Deliverable: A single reproducible experiment demonstrating the complete chain from correlation/statistics through the Wiener-Hopf solution to estimated signal, error, MSE, and visualization.

5. Module Connection

```
Config → [Module 1:  $x[n] \rightarrow h[n] \rightarrow y[n]$ ] →
[Module 2:  $R_{xx}/R_{xd} \rightarrow R, p \rightarrow w_{\text{opt}} \rightarrow \hat{d}[n] \rightarrow e[n] \rightarrow$ 
MSE] → Visualization
```

Each module must expose reusable outputs rather than isolated scripts. No module should depend on a hard-coded signal, experiment, or parameter set.

6. End-of-Phase Software Milestone

A single reproducible experiment must demonstrate the complete chain: discrete signal generation → discrete-time processing → correlation/statistics → Wiener-Hopf solution → estimated signal → error → MSE → visualization.

7. Forward Roadmap

This software foundation will be extended in subsequent phases toward Gradient Descent, LMS, NLMS, secondary-path modeling, FxLMS/FxNLMS, AI/hybrid processing, and real-time deployment.